Qualcomm Technologies, Inc.

SW6100 APSS GPIO Pin Control Software Reference

80-74841-92 Rev. AA

June 14, 2025

Qualcomm
Confidential - May Contain Trade Secrets
2025-12-29 05:39:58 GMT
hyungchul.won@lge.com

Confidential – Qualcomm Technologies, Inc. and/or its affiliated companies – May Contain Trade Secrets

NO PUBLIC DISCLOSURE PERMITTED: Please report postings of this document on public servers or websites to DocCtrlAgent@qualcomm.com.

© Qualcomm Technologies, Inc. and/or its subsidiaries. All rights reserved.

Revision history

Revision	Date	Description
AA	June 2025	Initial release

Qualcomm
Confidential - May Contain Trade Secrets
2025-12-29 05:39:58 GMT
hyungchul.won@lge.com

Contents

Revision history 2

1 APSS GPIO pin control overview 7

2 GPIO hardware overview 8

- 2.1 GPIO pin configuration and pin function 8
- 2.2 GPIO pin states 8
- 2.3 GPIO pad structure 8
- 2.4 GPIO configuration registers 10
- 2.5 GPIO input and output register 11
- 2.6 GPIO interrupt configuration register 11
- 2.7 GPIO interrupt status register 13
- 2.8 GPIO ID status register 13

3 GPIO TLMM v4 programming 14

- 3.1 Previous software model for GPIO programming 14
- 3.2 Software model for GPIO programming–TLMM v4 14

4 GPIO pin control software model 15

5 GPIO pin control software use cases 17

- 5.1 Use case 1–Pins driven by controllers 17
  - 5.1.1 Configure GPIO pins at bootup 17
  - 5.1.2 Configure GPIO pins for Active and Sleep states at runtime 18
  - 5.1.3 Configure GPIO pin groups with different function settings at runtime 20
  - 5.1.4 Configure GPIO pin groups for secure digital card pins at runtime 21
- 5.2 Use case 2–Pins driven by software (bit bang) 23
  - 5.2.1 Configure GPIO pins for software bit banging at runtime 24
- 5.3 Use case 3–Pins used as interrupt sources 26
  - 5.3.1 Configure GPIO pins as interrupt sources during bootup 26
- 5.4 Use case 4–Pins used as clock sources 27
  - 5.4.1 Configure GPIO pins as clock sources 27

6 GPIO debugging–adb commands 30

Qualcomm
Confidential – May Contain Trade Secrets
2025-12-29 05:59:58 GMT
hyungshul.won@lge.com

7 GPIO configuration in XBL 33
A References 37

Qualcomm
Confidential - May Contain Trade Secrets
2025-12-29 05:39:58 GMT
hyungchul.won@lge.com

Tables

Table 2-1: TLMM_GPIO_CFGn register bit field..... 10
Table 2-2: TLMM_GPIO_IN_OUTn register bit field..... 11
Table 2-3: TLMM_GPIO_INTR_CFGn register bit fields..... 11
Table 2-4: TLMM_GPIO_INTR_STATUS register bit field..... 13
Table 2-5: TLMM_GPIO_ID_STATUS register bit field.....13
Table 5-1: GPIO pin control software use cases..... 17
Table 7-1: Configuration properties for GPIO default values..... 33

Qualcomm
Confidential - May Contain Trade Secrets
2025-12-29 05:39:58 GMT
hyungchul.won@lge.com

Figures

Figure 2-1: Conceptual structure diagram of GPIO pad..... 9

Qualcomm
Confidential - May Contain Trade Secrets
2025-12-29 05:39:58 GMT
hyungchul.won@lge.com

1 APSS GPIO pin control overview

The Linux pin control framework enables you to configure the general purpose input/output (GPIOs) in the application processor subsystem (APSS). The framework is designed specifically for embedded applications on System-on-Chip (SoC) where the outfacing pins are considered precious and limited resources.

A pin multiplexing (PinMuxing) logic allows a pin or a group of pins to be reused for different purposes. Each use case of these pins requires different attributes to be configured to drive the signal on the pins. The pin control and PinMux framework allow users to specify different configuration attributes and define many uses or functions for a group of pins.

The document explains the pin control framework to configure the SoC pin resources on the device. It describes the different use cases for the SoC pins for this chipset.

The document is intended for Linux device driver developers and for software engineers who want to configure the GPIO in the APSS.

Conventions

Function declarations, function names, type declarations, attributes, and code samples appear in a different font, for example, `cp armcc armcpp`.

Code variables appear in angle brackets, for example, `<number>`.

Commands to be entered appear in a different font, for example, `copy a:*.* b:`.

Technical assistance

For help or clarification on information in this document, submit a case to Qualcomm Technologies, Inc. (QTI) athttps://www.qualcomm.com/support.

Qualcomm
Confidential - May Contain Trade Secrets
2025-12-29 05:39:58 GMT
hyungshul.wonolge.com

2 GPIO hardware overview

A total of 150 chip-level GPIOs and 23 low-power island (LPI) GPIOs and 46 are SSC GPIOs.

Module	Base address	Size	End address
LPASS_TLMM_GPIO	0x22360000	0x20000	0x22560000
TLMM_REG	0x0F100000	0xF00000	0x10000000
SSC_TLMM_GPIO	0x221C0000	0x30000	0x221F0000

2.1 GPIO pin configuration and pin function

GPIO pin configuration programming is an attribute that governs the nature of a signal to be driven on the lines. Typical configurations include drive strength, pull values, and direction. GPIO Pin function can be defined as the use of the signal to be driven on the GPIO pins.

2.2 GPIO pin states

The framework implementation uses the GPIO pin states. A state comprises configurations and functions to be applied on a group of pins. An owner of one or more pins must furnish the information that comprises the state of a pin group. The framework allows competing owners, that is, independent clients that want to use the same GPIO pins for different purposes can furnish the respective GPIO pin state information.

There is no limit to the number of states an owner can define for a GPIO pin. However, owners typically define the following three states:

- Default
- Active
- Sleep

For a detailed description of pin control framework and client APIs, see PINCTRL (Pin Control) Subsystem.

2.3 GPIO pad structure

A GPIO pin can be configured as general input and output, or one of the several alternate functions. When a GPIO pin is used as an input it can also be set as an interrupt source.

Qualcomm Confidential – May Contain Trade Secrets 2025-12-29 07:29:56 GMT hyungchul.won@qti.com

A subset of GPIOs can generate interrupts that can bring the mobile device out of deep sleep (XO shutdown or VDD minimization). GPIOs can also be configured to have internal pull circuits (up, down, or no-pull) and different driving strengths.

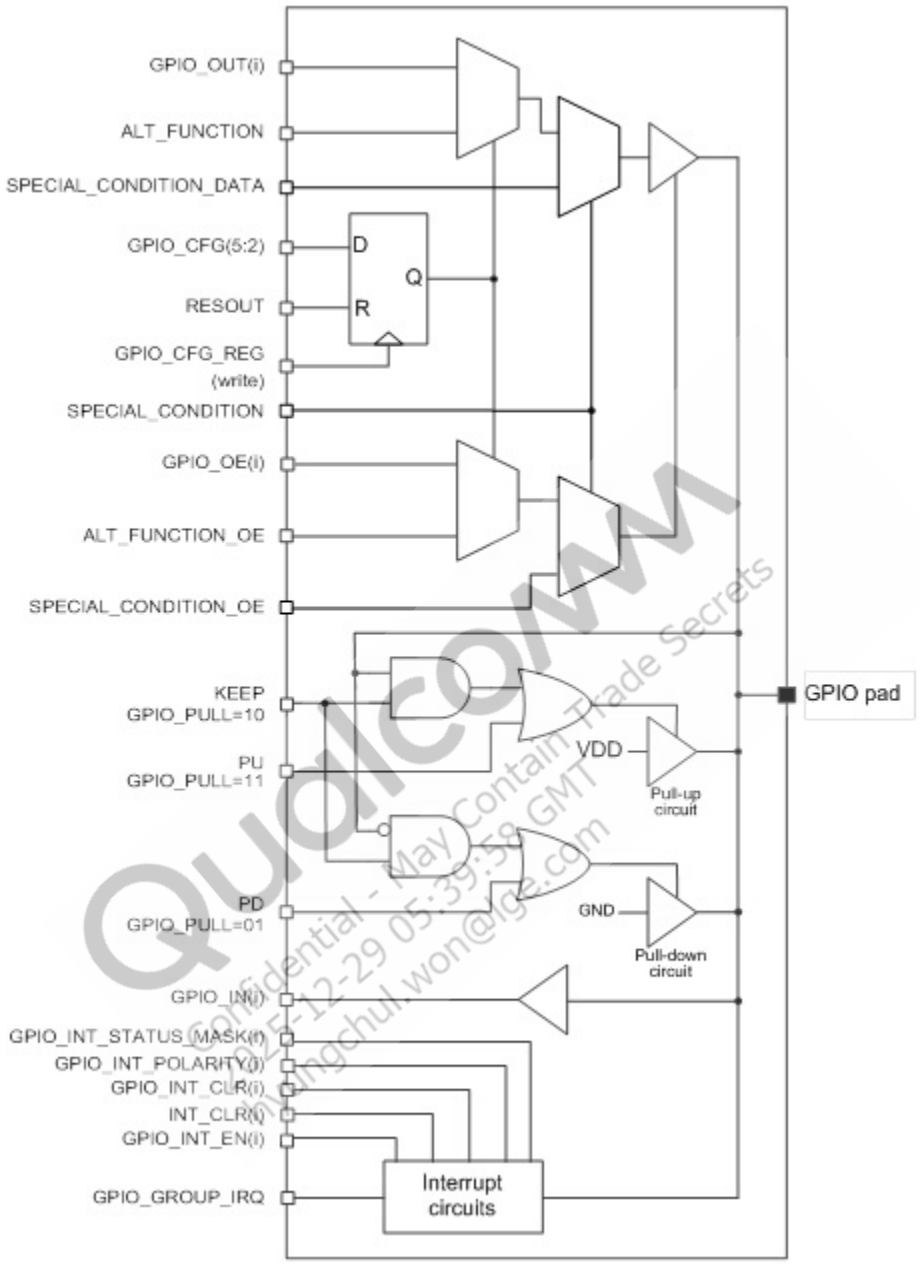

Figure 2-1 Conceptual structure diagram of GPIO pad

Qualcomm Confidential - May Contain Trade Secrets 2025-12-29 05:39:58 GMT jongchul.won@ige.com

2.4 GPIO configuration registers

The TLMM_GPIO_CFGn, n = 0…149 register control the configuration of GPIOs. Each register controls the configuration of one GPIO.

Table 2-1 TLMM_GPIO_CFGn register bit field

Bit	Name	Description
13	I2C_PULL	This bit indicates I2C pull configuration 1–Pull-up 0–Pull-down
12	EGPIO_ENABLE	Controls EGPIO functionality on this GPIO 1–Enable EGPIO 0–Disable EGPIO
11	EGPIO_PRESENT	Indicates availability of EGPIO feature on this GPIO ■ 1–Available ■ 0–Not available
10	GPIO_HIHYS_EN	Controls the hysteresis of GPIO[n] on input modes
9	GPIO_OE	Controls the output enable (OE) for GPIO[n] when it's in GPIO mode
8:6	DRV_STRENGTH	Controls the GPIO pad drive strength; this applies regardless of the FUNC_SEL field selection: ■ 000–2 mA ■ 001–4 mA ■ 010–6 mA ■ 011–8 mA ■ 100–10 mA ■ 101–12 mA ■ 110–14 mA ■ 111–16 mA
5:2	FUNC_SEL	Many of the GPIO pads have one or more functional hardware interfaces. This field controls how the pad is used. Set this field to 0 for GPIO mode.
1:0	GPIO_PULL	The pad can be configured to use an internal weak pull-up, pull-down, keeper, or no-pull function; this applies regardless of the FUNC_SEL field selection: ■ 00–No-pull ■ 01–Pull-down (PD) ■ 10–Keeper ■ 11–Pull-up (PU)

Qualcomm Confidential - May Contain Trade Secrets 2025-12-29 05:39:58 GMT hyungchul.won@lge.com

2.5 GPIO input and output register

When the FUNC_SEL bit of a GPIO is set to 0, the GPIO is used as a general purpose pin. The GPIO_OE bit of the TLMM_GPIO_CFGn register controls the value of the bit:

- If set to 1, the pin is used as an output pin and bit 1 of the TLMM_GPIO_IN_OUTn, n = 0…149 register controls its state
- If set to 0, the pin is used as an input pin. The state of a GPIO can be read from bit 0 in the same register

Table 2-2 TLMM_GPIO_IN_OUTn register bit field

Bit	Name	Description
31:2	Reserved	Reserved field.
1	GPIO_OUT	Controls the output state of an output pin. ■ 0–Low ■ 1–High
0	GPIO_IN	Reads the state of an input pin.

2.6 GPIO interrupt configuration register

All GPIOs can be set as interrupt sources. The TLMM_GPIO_INTR_CFGn, n = 0...149 register control the GPIO interrupt functionality.

Table 2-3 TLMM_GPIO_INTR_CFGn register bit fields

Bit	Name	Description
31:9	Reserved	Reserved field
10:8	TARGET_PROC	Determines the processor to which a summary interrupt from GPIO[n] gets routed: ■ 0x0–Sensors: Routes the GPIO[n] signal to the sensors summary interrupt ■ 0x1–LPA_DSP: Routes the GPIO[n] signal to the LPASS summary interrupt ■ 0x2–RPM: Routes the GPIO[n] signal to the RPM summary interrupt ■ 0x3–APSS: Routes the GPIO[n] signal to the APSS summary interrupt ■ 0x4–MSS: Routes the GPIO[n] signal to the MSS summary interrupt ■ 0x5–TrustZone: Routes the GPIO[n] signal to the TrustZone summary interrupt ■ 0x6–cDSP: Routes the GPIO[n] signal to the cDSP summary interrupt ■ 0x7–None: Doesn't route to any processor subsystem; this is the default setting

Qualcomm Confidential - May Contain Trade Secrets 2025-12-29 05:39:58 GMT hyungchul.won@lge.com

Table 2-3 TLMM_GPIO_INTR_CFGn register bit fields (cont.)

Bit	Name	Description
7	WAKEUP_ENABLE	Enables a GPIO to generate wakeup-capable interrupts ■ 1–Enable ■ 0–Disable
6	WAKEUP_PRESENT	Determines If this GPIO is wakeup-capable
5	DIR_CONN_EN	Notifies the top-level module multiplex (TLMM) that GPIO[n] is being used as a direct connect interrupt: ■ 0–Disables the GPIO as a direct connect interrupt ■ 1–Enables the GPIO as a direct connect interrupt
4	INTR_RAW_STATUS_EN	Enables the RAW status for the summary interrupt on this GPIO; this is a power-saving mechanism and should be left disabled unless necessary: ■ 1–Enable ■ 0–Disable
3:2	INTR_DECT_CTL	Controls the edge or level detection of the interrupt controller: ■ 0x0–LEVEL: Level sensitive ■ 0x1–POS_EDGE: Positive edge sensitive ■ 0x2–NEG_EDGE: Negative edge sensitive ■ 0x3–DUAL_EDGE: Sensitive to both edges
1	INTR_POL_CTL	Controls the polarity detection of the interrupt controller: ■ Polarity 1 corresponds to active high ■ Polarity 0 corresponds to active low ▫ 0–POLARITY_0 ▫ 1–POLATRITY_1
0	INTR_ENABLE	Controls the generation of summary interrupt: ■ 1–Enable ■ 0–Disable

The trigger type of the interrupt is controlled by INTR_POL_CTL and INTR_DETC_CTL. Setting or clearing the INTR_ENABLE bit field enables or disables the corresponding GPIO interrupt.

Qualcomm Confidential - May Contain Trade Secrets 2025-12-29 05:39:58 GMT hyungchul.won@lge.com

2.7 GPIO interrupt status register

The interrupt status for GPIO[n] is recorded in the TLMM_GPIO_INTR_STATUSn, n = 0 ... 149 register.

Table 2-4 TLMM_GPIO_INTR_STATUS register bit field

Bit	Name	Description
31:1	Reserved	Reserved field
0	INTR_STATUS	When read, returns the interrupt status of GPIO[n] ■ 0–The interrupt isn't active ■ 1–The interrupt is active When written, clears or sets the interrupt for GPIO[n] ■ 0–Clear the interrupt ■ 1–Set the interrupt

2.8 GPIO ID status register

The ID status register identifies the tile in which GPIO ID is present.

The GPIO ID status for GPIO[n] is recorded in the TLMM_GPIO_ID_STATUSn, n = 0 ... 149 register. When read, this register returns the ID status of the corresponding GPIO.

- The GPIO belongs to particular tile when the register returns 1.
- When the register returns 0, GPIO doesn't belong to that tile.

Table 2-5 TLMM_GPIO_ID_STATUS register bit field

Bits	Name	Description
31:1	Reserved	Reserved field
0	GPIO_ID_STATUS	Read the state of an ID pin

Qualcomm Confidential - May Contain Trade Secrets 2025-12-29 05:39:58 GMT hyungchul.kim@lge.com

3 GPIO TLMM v4 programming

SoCs have a PinMux controller known as TLMM. The current version of TLMM is v4.

3.1 Previous software model for GPIO programming

Prior to TLMM v4, the GPIO library was used to configure pins and apply specific functions to them. Clients defined at least two sets of configurations, Active and Sleep states.

The `gpio_request()` and `gpio_free()` APIs were used to install and revert the Active and Sleep pin configuration states at runtime. In few cases, a separate API was provided to the clients to program more than two configurations.

3.2 Software model for GPIO programming–TLMM v4

TLMM v4 supports multiplexing and configuration of different types of pins. Any SoC that incorporates TLMM v4 controller can only have pins of types supported by TLMM v4.

These types have different register semantics, also have their own protocol-specific pin attributes in certain cases (attributes different from drive strength, pull, and direction).

In the context of pin control and based on the use cases (pins driven by controllers, software, clocks, and interrupts), the guide applies to the GPIO supported by TLMM v4.

Qualcomm
Confidential – May Contain Trade Secrets
2025-12-29 05:19:35 EST
hyungchul.won@qc.com

4 GPIO pin control software model

This section provides the features of a GPIO pin-based model.

- All pin configurations and TLMM information are defined in an SoC-specific file, `vienna-pinctrl.dtsi`. This file has the pin groupings and the configurations and functions to be applied to them.
- Individual client device tree nodes reference the configuration nodes defined in `vienna-pinctrl.dtsi`. It consists the state of pins. A client can have any number of configuration nodes in a state, and any number of states.
- The state information of a GPIO is parsed with a call to `devm_pinctrl_get()`.
- A client can then choose to install a given state using `pinctrl_lookup_state()` and `pinctrl_select_state()`.

In the following example, `arch/arm/boot/dts/qcom/vienna-pinctrl.dtsi` is a sample TLMM PinMux node containing the pin types:

```
&soc {

    tlmm: pinctrl@f000000 {
        compatible = "qcom,vienna-pinctrl";
        reg = <0x0F000000 0x1000000>;
        interrupts = <GIC_SPI 208 IRQ_TYPE_LEVEL_HIGH>;
        gpio-controller;
        #gpio-cells = <2>;
        interrupt-controller;
        #interrupt-cells = <2>;
        wakeup-parent = <&pdc>;
        qcom,gpios-reserved = <0 1 2 3 14 15 73 74>;

        qupv3_se0_i2c_pins: qupv3_se0_i2c_pins {
            qupv3_se0_i2c_sda_active: qupv3_se0_i2c_sda_active {
                mux {
                    pins = "gpio0";
                    function = "qup0_se0_l0";
                };

                config {
                    pins = "gpio0";
                    drive-strength = <2>;
                    bias-pull-up;
```

Qualcomm Confidential – May Contain Trade Secrets 2025-12-29 05:32:58 GMT shyong@hul.wor.@ge.com

```
                qcom,i2c_pull;
            };
        };

        qupv3_se0_i2c_scl_active: qupv3_se0_i2c_scl_active {
            mux {
                pins = "gpio1";
                function = "qup0_se0_l1";
            };

            config {
                pins = "gpio1";
                drive-strength = <2>;
                bias-pull-up;
                qcom,i2c_pull;
            };
        };

        qupv3_se0_i2c_sleep: qupv3_se0_i2c_sleep {
            mux {
                pins = "gpio0", "gpio1";
                function = "gpio";
            };

            config {
                pins = "gpio0", "gpio1";
                drive-strength = <2>;
            };
        };
    };
```

Qualcomm Confidential - May Contain Trade Secrets 2025-12-29 05:39:58 GMT hyungchul.won@lge.com

5 GPIO pin control software use cases

The table lists a few use cases where the pins on SoCs can be used.

Table 5-1 GPIO pin control software use cases

Use case	Description
Use case 1	Pins driven by controllers. For example, universal asynchronous receiver/transmitter (UART), serial peripheral interface (SPI), and inter-integrated circuit (I^2C).
Use case 2	Pins used by software for bit banging; typically the devices attached externally to the board, for example, display panels.
Use case 3	Pins that are used as a source of interrupts. They're attached externally to the board. For example, Ethernet.
Use case 4	Pins used as clock sources.

In a few instances, OEMs use the same group of pins for use cases 1 and 2. In such instances, these two cases are modeled as extra states in the device tree.

WARNING The pinctrl framework limits the context in which pinctrl APIs can be invoked. If the driver violates the context constraints, redesign the corresponding part. For information about the constraints, see PINCTRL (Pin Control) Subsystem.

5.1 Use case 1–Pins driven by controllers

This use case uses the example provided in GPIO pin control software model.

5.1.1 Configure GPIO pins at bootup

1. Add a pin grouping node to the `vienna-pinctrl.dtsi` file.

```
&soc {
     tlmm: pinctrl@f000000 {
{
…..
….
..
Client1_pins {
            Client1_default: client1_default{
                mux {
                    pins = "gpio54";
                    function = "gpio";
```

```
                    };

                    config {
                        pins = "gpio54";
                        drive-strength = <2>; /* 2 mA */
                        bias-pull-down; /* pull down */
                        input-enable;
                    };
                };
            };
};
```

2. Change the client node in the device tree (`vienna-<board>.dtsi`).

```
Soc {
…….
…
..
Client1 {
……
…
/*Note the default state is programmed by the kernel at the time of kernel
bootup. No driver changes are necessary since at probe time the  default
state would already be programmed in TLMM */
pinctrl-states = "default";
/* Use phandle reference to default configuration node */
pinctrl-0 = <&Client1_default>;
};
```

5.1.2 Configure GPIO pins for Active and Sleep states at runtime

In this case, a controller driver has Active state for pins during transactions or as the default state, and Sleep state when suspending the system. The driver must install the correct state accordingly.

To configure general-purpose pins for Active and Sleep states:

1. Define the pin group node containing Active and Sleep states.

```
&soc {
tlmm: pinctrl@f000000 {
…..
….
/* I2C CONFIGURATION */


qupv3_se1_i2c_pins: qupv3_se1_i2c_pins {
            qupv3_se1_i2c_sda_active: qupv3_se1_i2c_sda_active {
                mux {
                    pins = "gpio4";
                    function = "qup0_se1_l0";
                };
```

```
            config {
                pins = "gpio4";
                drive-strength = <2>;
                bias-pull-up;
                qcom,i2c_pull;
            };
        };

        qupv3_se1_i2c_scl_active: qupv3_se1_i2c_scl_active {
            mux {
                pins = "gpio5";
                function = "qup0_se1_l1";
            };

            config {
                pins = "gpio5";
                drive-strength = <2>;
                bias-pull-up;
                qcom,i2c_pull;
            };
        };

        qupv3_se1_i2c_sleep: qupv3_se1_i2c_sleep {
            mux {
                pins = "gpio4", "gpio5";
                function = "gpio";
            };

            config {
                pins = "gpio4", "gpio5";
                drive-strength = <2>;
            };
        };
    };
```

2. Change the client node to include pinctrl states. Delete any explicit references to the GPIOs using the GPIO chip node.

```
Soc {
…….
…
Client2 {
……
        pinctrl-states = “active”, “suspend”;
        /* Use phandle referrence to active and sleep configurations to
        Define the active and sleep pin states */
             pinctrl-0 =
<&qupv3_se1_i2c_sda_active>,<&qupv3_se1_i2c_scl_active>;
```

Qualcomm Confidential - May Contain Trade Secrets 2025-12-29 05:39:58 GMT hyungchul.won@lge.com

```
                pinctrl-1 = <&qupv3_se1_i2c_sleep>;
};
```

3. Change the driver to the pinctrl interface.

```
Static int client2_probe (struct platform_device *pdev) {
        Struct pinctrl_state *set_state;
        Struct client2_data *client2_dd;
        …..
        …
        ..
        /* Try to obtain pinctrl handle */
        Pdev->dev->pins->p = devm_pinctrl_get(pdev);
        /* Lookup the active configuration */ 34
        set_state = pinctrl_lookup_state(pdev->dev->pins->p, "active");
        if (!set_state)
            goto fail;
        else
            /*Actually write the active configuration to hardware */
        ret = pinctrl_select_state(pdev->dev->pins->p, set_state);

        /* Install sleep configuration in runtime suspend function */
}

Static int client2_suspend( struct platfom_device) {
        /* Configure pins to sleep state state */
        Struct pinctrl_state *set_state;
        /* Actually write the sleep configuration to hardware */
        set_state = pinctrl_lookup_state(pdev->dev->pins->p, "sleep");
        if (!set_state)
            goto fail;
        else
        ret = pinctrl_select_state(pdev->dev->pins->p, set_state);
}
```

Qualcomm Confidential - May Contain Trade Secrets 2025-12-29 05:39:58 GMT kwungchul.won@lge.com

5.1.3 Configure GPIO pin groups with different function settings at runtime

The function applied on a group of pins remains unchanged for the use cases discussed until now. However, in few use cases, the function applied to a pin group changes as part of installing a new state. In these cases, the pin group is modeled as two separate pin groups using the same pins.

The driver chooses to install a new state for the pin groups with different or same configurations, however, with different pin functions. The following example shows different pin functions applied to the same pin group:

```
&soc {
tlmm: pinctrl@ f000000 {
...
..
    qupv3_se0_4uart_pins: qupv3_se0_4uart_pins {
            qupv3_se0_default_cts: qupv3_se0_default_cts {
```

```
        mux {
            pins = "gpio0";
            function = "gpio";
        };

        config {
            pins = "gpio0";
            drive-strength = <2>;
            bias-disable;
        };
    };

    qupv3_se0_default_rts: qupv3_se0_default_rts {
        mux {
            pins = "gpio1";
            function = "gpio";
        };

        config {
            pins = "gpio1";
            drive-strength = <2>;
            bias-pull-down;
        };
    };

    qupv3_se0_default_tx: qupv3_se0_default_tx {
        mux {
            pins = "gpio2";
            function = "gpio";
        };

        config {
            pins = "gpio2";
            drive-strength = <2>;
            bias-pull-up;
        };
    };
```

Qualcomm Confidential - May Contain Trade Secrets 2025-12-29 05:35:58 GMT hyungchul.won@rse.com

5.1.4 Configure GPIO pin groups for secure digital card pins at runtime

TLMM v4 supports dedicated secure digital card (SDC) pin configurations for SDC1 and SDC2. These pins are used for clock, command, and data signals as part of the SDC specification.

Qualcomm models this as six SDC type pins:

- Pins 0 to 2 correspond to clock, command, and data of SDC1.
- Pins 3 to 5 correspond to clock, command, and data of SDC2.

The following example describes the configuration of SDC pin types:

1. Extend the PinMux node to define pins for SDC1 and SDC2.

```
&Soc {
          tlmm: pinctrl@ f000000 {
...
..
        /* SDC pin type */

            sdc1_on: sdc1_on {
            clk {
                pins = "sdc1_clk";
                bias-disable;
                drive-strength = <16>;
            };


            cmd {
                pins = "sdc1_cmd";
                bias-pull-up;
                drive-strength = <10>;
            };


            data {
                pins = "sdc1_data";
                bias-pull-up;
                drive-strength = <10>;
            };

            rclk {
                pins = "sdc1_rclk";
                bias-pull-down;
                drive-strength = <2>;
            };
        };

        sdc1_off: sdc1_off {
            clk {
                pins = "sdc1_clk";
                drive-strength = <2>;
                bias-bus-hold;
            };

            cmd {
                pins = "sdc1_cmd";
                drive-strength = <2>;
                bias-bus-hold;
```

Qualcomm Confidential - May Contain Trade Secrets 2025-12-29 05:39:58 GMT hyungchul.yoon@lge.com

```
            };

            data {
                pins = "sdc1_data";
                drive-strength = <2>;
                bias-bus-hold;
            };

            rclk {
                pins = "sdc1_rclk";
                drive-strength = <2>;
                bias-bus-hold;
            };
        };
```

2. Update the secure digital high capacity (SDHC) nodes to point to the pin configurations defined in Step 1. Delete the old configurations and add your configuration in the `vienna-<board>.dtsi`.

```
&sdhc_1 {
        …..
        …
pinctrl-names = "active", "sleep";
       pinctrl-0 = <&sdc1_on>;
       pinctrl-1 = <&sdc1_off>;
};
```

3. Update the driver to parse the device tree for pinctrl nodes as shown in Step 3 of Configure GPIO pins for Active and Sleep states at runtime.

5.2 Use case 2–Pins driven by software (bit bang)

The following two frameworks apply to this use case:

- Pinctrl to configure the pin
- GPIO lib to drive the required signal values on the pin

The general-purpose pin type on the TLMM node (currently the only pin type on TLMM v4 to support GPIO functionality) must be augmented with a GPIO chip controller node. Extend the TLMM PinMux node from GPIO pin control software model, as shown in Configure GPIO pins for software bit banging at runtime. The GPIO controller node is used as a handle to specify GPIO pins in client nodes. Such pins are typically used outside the SoC, for example, display panel, Ethernet. These pin group nodes are added by extending the tlmm_pinmux node in the corresponding `<soc>-board.dtsi` file.

Configure GPIO pins for software bit banging at runtime shows configuration of the TLMM PinMux node with the GPIO controller for the general-purpose pin type.

Qualcomm Confidential - May Contain Trade Secrets 2025-12-23 05:35:58 GMT hyungshul.woo@lge.com

5.2.1 Configure GPIO pins for software bit banging at runtime

The following example shows GPIO client and pin configuration for Active and Sleep states:

1. Extend the `tlmm_pinmux` node in the corresponding `vienna-pinctrl.dtsi` file to add pin configuration node for Active and Sleep states.

```
&soc {
tlmm: pinctrl@ f000000 {
…..
….
Client3_pins {
/* Pin group node */
/* Uses general purpose pins */
            cdc_reset_sleep: cdc_reset_sleep {
                mux {
                    pins = "gpio64";
                    function = "gpio";
                };
                config {
                    pins = "gpio64";
                    drive-strength = <16>;
                    bias-disable;
                    output-low;
                };
            };
            cdc_reset_active:cdc_reset_active {
                mux {
                    pins = "gpio64";
                    function = "gpio";
                };
                config {
                    pins = "gpio64";
                    drive-strength = <16>;
                    bias-pull-down;
                    output-high;
                };
            };
        };

};
```

2. Change the client node to include pinctrl states. Also, change any explicit references to the GPIO chip handler from Use case 2–Pins driven by software (bit bang).

```
Soc {
…….
…
Client3 {
……
```

Qualcomm Confidential - May Contain Trade Secrets 2025-12-29 05:39:58 GMT hyungchul.won@lge.com

```
        pinctrl-states = "active", "sleep";
        /* Use phandle referrence to active and sleep configurations to
Define the active and sleep pin states */
        pinctrl-0 = <& cdc_reset_active >;
        pinctrl-1 = <& cdc_reset_sleep >;
        /* Use phandle of correct gpio controller */
        client3-gpio-1 = <&tlmm 64 0>;
};
```

3. Change the driver to look for pinctrl states only. The GPIOs are probed even after the pinctrl states are found.

```
int client3_probe ( struct platform_device *pdev)
{
        struct pinctrl_state *set_state;
        …….
        ….
        /* Try to obtain pinctrl handle */ 30
        pdev->dev->pins->p = devm_pinctrl_get(pdev);
        If (!pdev->dev->pins->p) {
        /* Lookup the pin configuration */
        set_state = pinctrl_lookup_state(pdev->dev->pins->p, "active");
        if (!set_state)
            goto fail;
        client3_data->active_state = set_state;
        set_state = pinctrl_lookup_state(pdev->dev->pins->p, "suspend");
        client3_data->sleep_state = set_state;
        /*(Optional) Actually write the active configuration to hardware*/
        ret = pinctrl_select_state(pdev->dev->pins->p,client3_data-
>active_state);
}
gpio_probe:
        /* (Perform Regular probe of gpios. */
    Client3_data->reset_gpio = of_get_named_gpio(node, "client3_gpio_1");
        Gpio_request(Client3_data->reset_gpio, "reset");
…..
}
```

4. Install the required configuration before bit banging.

```
int Client3_drive_signal ( boolean active )
{
        If (client3_data->use_pinctrl) {
            If (active)
                ret = pinctrl_select_state(pdev->dev->pins->p,
            client3_data->active_state);
            else
                ret = pinctrl_select_state(pdev->dev->pins->p,
            client3_data->sleep_state);
        }
```

Qualcomm Confidential - May Contain Trade Secrets 2025-12-29 05:39:39 GMT hyungsul.won@lge.com

```
        gpio_direction_out(client3_data->reset_gpio);
        gpio_set_value(client3_data->reset_gpio, 1);
}
```

5.3 Use case 3–Pins used as interrupt sources

The general-purpose pins present on a TLMM block can be used as interrupt sources. The PinMux node must be augmented with interrupt controller attributes to support this use case. Since this is supported by general-purpose pin types, augment the corresponding GPIO controller node with interrupt controller attributes.

Configure GPIO pins as interrupt sources during bootup shows the PinMux node with interrupt controller attributes.

5.3.1 Configure GPIO pins as interrupt sources during bootup

When pins are used as interrupt sources, the pin attributes are configured once, at bootup. The pin is then used as an interrupt source. To do this, the pin configuration must be modeled as a default state of the pin. Also, since these devices are present as attachments to the SoC on a board, their pin groups are added by extending the PinMux node in the corresponding board-specific device tree source file.

The following example shows an interrupt request (IRQ) client:

1. Extend the `tlmm_pinmux` node in the `vienna-pinctrl.dtsi` file to add the pin configuration.

```
&soc {
tlmm: pinctrl@ f000000 {
…..
….
Client4_pins {
        Client4_default: client4_default{
            mux {
                pins = "gpio54";
                function = "gpio";
            };

            config {
                pins = "gpio54";
                drive-strength = <2>; /* 2 mA */
                bias-pull-down; /* pull down */
                input-enable;
            };
        };
    };
};
```

2. Update the client device node to point to the pin configuration node.

```
Soc {
…….
…
```

Qualcomm Confidential May Contain Trade Secrets 2025-12-29 05:39:58 GMT hyungchul.wan@lge.com

```
Client4 {
……
        pinctrl-states = "default";
        /* Use phandle referrence to default config node above */
        pinctrl-0 = <&client4_default>;
        interrupt-parent = <&tlmm>;
        /* Add reference to the interrupt controller node defined in
example above
        Interrupts = <54 0>;
          Client4, irq-gpio = <&tlmm 54 0x00>;

};
```

3. Driver changes to apply pin configuration isn't needed as default pin configuration is installed by the kernel at bootup.

```
/* (Perform Regular probe of gpios. */
        Client4_data->gpio_irq = of_get_named_gpio(node,
"client4_gpio_1");
        Gpio_request(Client4_data->gpio_irq, "clinet4");

    ret = request_threaded_irq(gpio_to_irq(Client4_data->gpio_irq), NULL,
client4_irq_thread, IRQF_TRIGGER_LOW|IRQF_ONESHOT|IRQF_TRIGGER_HIGH,
"clinet4", Client4_data->gpio_irq);
…..
}
```

5.4 Use case 4–Pins used as clock sources

Few general-purpose pins present on a TLMM block can be used as clock sources. The PinMux node must be augmented with clock controller attributes to support this use case. Since this is supported by GPIO pin types, augment the corresponding GPIO controller node with clock controller attributes.

Configure GPIO pins as clock sources shows the PinMux node with clock controller attributes.

5.4.1 Configure GPIO pins as clock sources

When pins are used as clock sources, the pin attributes are configured once at bootup or configured dynamically. If the pin needs to be configured at bootup only, then the pin configuration must be modeled as a default state of the pin, otherwise it can define active and sleep configurations.

Also, since these devices are present as attachments to the SoC on a board, their pin groups are added by extending the PinMux node in the corresponding board-specific device tree source file.

The following example shows a clock client:

1. Extend the `tlmm_pinmux` node in the `vienna-pinctrl.dtsi` file to add the pin configuration.

```
&soc {
tlmm: pinctrl@ f000000 {
…..
….
Client5_pins {
```

```
                Client5_active: client5_active{
                    mux {
                        pins = "gpio8";
                        function = " gcc_gp1 ";
                    };

                    config {
                        pins = "gpio57";
                        drive-strength = <2>; /* 2 mA */
                        bias-pull-none; /* pull none */
                    };
                };
                Client5_sleep: client5_sleep{
                    mux {
                        pins = "gpio8";
                        function = "gpio";
                    };

                    config {
                        pins = "gpio57";
                        drive-strength = <2>; /* 2 mA */
                        bias-pull-down; /* pull down */
                    };
                };

            };
};
```

2. Update the client device node to point to the pin configuration node.

```
Soc {
…….
…
Client5 {
    ……
    pinctrl-states = “active”, “suspend”;
    /* Use phandle referrence to active and sleep configurations to
    Define the active and sleep pin states */
    pinctrl-0 = <& Client5_active >;
    pinctrl-1 = <& Client5_sleep>;
    clocks = <&clock_gcc gp1_clk_src>,
    clock-names = "gp1_clk";
};

drivers/clk/qcom/gcc-vienna. c
static const struct freq_tbl ftbl_gcc_gp1_clk_src[] = {
    F(25000000, P_GCC_GPLL0_OUT_EVEN, 12, 0, 0),
    F(50000000, P_GCC_GPLL0_OUT_EVEN, 6, 0, 0),
    F(100000000, P_GCC_GPLL0_OUT_MAIN, 6, 0, 0),
```

Qualcomm Confidential - May Contain Trade Secrets 2025-12-29 05:35:58 GMT hyungchul.won@lge.com

```
    F(200000000, P_GCC_GPLL0_OUT_MAIN, 3, 0, 0),
    + F(1000, P_BI_TCXO, 16, 1, 1200),
};

// You need use this test code changes in probe function
int client5_probe ( struct platform_device *pdev)
{
struct pinctrl_state *set_state;
struct clk *pwm_clk = NULL;
int ret;
…….
….
/* Try to obtain pinctrl handle */ 30
pdev->dev->pins->p = devm_pinctrl_get(pdev);
If (!pdev->dev->pins->p) {
/* Lookup the pin configuration */
set_state = pinctrl_lookup_state(pdev->dev->pins->p, “active”);
if (!set_state)
goto fail;
client5_data->active_state = set_state;
set_state = pinctrl_lookup_state(pdev->dev->pins->p, “suspend”);
client5_data->sleep_state = set_state;
Actually write the active configuration to hardware*/
ret = pinctrl_select_state(pdev->dev->pins->p,
client5_data->active_state);
pwm_clk = clk_get(dev, "gp1-clk");
if (IS_ERR(pwm_clk)) {
dev_err(dev, "%s: Get pwm_clk error\n", __func__);
return -ENODEV;
}

// Change the clock frequency by using the below piece of code
ret = clk_set_rate(pwm_clk, (unsigned long)10000);
if (ret < 0) {
dev_err(dev, "%s: Can't set pwm_clk rate 10KHz (%d)\n",
__func__, ret);
clk_put(pwm_clk);
return ret;
}

ret = clk_prepare_enable(pwm_clk);
if (ret < 0) {
dev_err(dev, "%s: Can't enable pwm_clk %d\n",
__func__, ret);
clk_put(pwm_clk);
return ret;
}
```

Qualcomm Confidential - May Contain Trade Secrets 2025-12-29 05:39:58 GMT hyungchul.won@lge.com

6 GPIO debugging–adb commands

The legacy **sysfs GPIO interface** has been deprecated in favor of the **libgpiod** (GPIO device) interface, which provides a more robust and flexible way to control GPIOs from user space. This document outlines how to use libgpiod tools and APIs to manage GPIO lines on Linux systems.

Linux exposes GPIO controllers as character devices under `/dev/`. Each controller is represented as a gpiochipN device.

For more information about gpiolibd, see Using GPIO Lines in Linux—The Linux Kernel documentation.

Download and compile libgpiod

Install the ARM64 toolchain and run the following commands on the host computer:

```
sudo apt install gcc-aarch64-linux-gnu
sudo apt install binutils-aarch64-linux-gnu
wget https://www.kernel.org/pub/software/libs/libgpiod/libgpiod-1.6.4.tar.xz
tar xvf libgpiod-1.6.4.tar.xz
cd libgpiod-1.6.4
```

Configure sources

Create dynamic libraries:

```
./configure --enable-tools=yes --build x86_64-pc-linux-gnu --host aarch64-
linux-gnu CFLAGS="-static -static-libgcc -Wl,-static,--start-group,/usr/lib/
gcc-cross/aarch64-linux-gnu/7.5.0/libgcc.a,/usr/lib/gcc-cross/aarch64-linux-
gnu/7.5.0/libgcc_eh.a,/usr/aarch64-linux-gnu/lib/libc.a,--end-group"
make
```

Qualcomm Confidential - May Contain Trade Secrets 2025-12-29 05:39:58 GMT hyunchul.won@lge.com

Build statically linked libraries:

```
aarch64-linux-gnu-gcc -static -o tools/gpiodetect tools/gpiodetect.o tools/
tools-common.o -Wl,-L<ABSOLUTE_PATH_TO_LIBGPIOD>libgpiod-1.6.4/lib/.libs,-
lgpiod,-lpthread,-static
```

```
aarch64-linux-gnu-gcc -static -o tools/gpioget tools/gpioget.o tools/tools-
common.o -Wl,-L<ABSOLUTE_PATH_TO_LIBGPIOD>/libgpiod-1.6.4/lib/.libs,-lgpiod,-
lpthread,-static
```

```
aarch64-linux-gnu-gcc -static -o tools/gpioset tools/gpioset.o tools/tools-
common.o -Wl,-L<ABSOLUTE_PATH_TO_LIBGPIOD>/libgpiod-1.6.4/lib/.libs,-lgpiod,-
lpthread,-static
```

```
aarch64-linux-gnu-gcc -static -o tools/gpiofind tools/gpiofind.o tools/tools-
common.o -Wl,-L<ABSOLUTE_PATH_TO_LIBGPIOD>/libgpiod-1.6.4/lib/.libs,-lgpiod,-
lpthread,-static
```

```
aarch64-linux-gnu-gcc -static -o tools/gpiomon tools/gpiomon.o tools/tools-
common.o -Wl,-L<ABSOLUTE_PATH_TO_LIBGPIOD>/libgpiod-1.6.4/lib/.libs,-lgpiod,-
lpthread,-static
```

After the compilation is complete, push these binaries to the device:

```
adb push gpiodetect gpioinfo gpioset gpioget /tmp
```

List the GPIO chips and lines

- To list all the GPIO chips available on the device, use `./gpiodetect`.
- To list all the lines in a GPIO chip, use `./gpioinfo gpiochip4`.

Set GPIO as output and drive value

Use `gpioset` to set a GPIO value and direction as output.

- `./gpioset gpiochip4 1 1`: Set the direction of GPIO1 in gpiochip4 as output and its value is set to 1.
- `./gpioset gpiochip4 1 0`: Set the direction of GPIO1 in gpiochip4 as output and its value is set to 0.

Set GPIO as input and read its value

Use `gpioget` to read a GPIO value and set direction as input.

`./gpioget gpiochip4 1` sets the direction of GPIO1 in gpiochip4 as the input and reads the current value.

GPIO configuration using devmem

Devmem is used to access and manipulate the GPIO registers in the Linux systems. This allows developers to read, and write to the specific GPIO memory addresses, enabling precise control and debugging of GPIO pin configurations.

Qualcomm Confidential – May Contain Trade Secrets 2025-12-29 05:39:58 GMT hyungchul.wo@lge.com

Run the following commands to compile devmem on the host computer machine and pushing it as a static library:

```
sudo apt install gcc-arm-linux-gnueabi gcc-arm-linux-gnueabihf

wget https://busybox.net/downloads/busybox-1.36.1.tar.bz2

tar xjf busybox-1.36.1.tar.bz2

cd busybox-1.36.1

make defconfig

make menuconfig
Settings --> Build Static Binary (No Shared Libraries) --> We need to enable
this
Settings --> cross compiler prefix(arm-linux-gnueabi-)

make -j$(nproc)

cp busybox devmem

adb push devmem /tmp/
```

Enable/disable devmem configurations

- To enable the configuration, use CONFIG_DEVMEM.
- To disable the configuration, use CONFIG_STRICT_DEVMEM.

Read/write devmem address

- To read an address, use `./devmem <address>`.

  For example: `./devmem 0xF100000`
- To write an address, use `./devmem <address> <size> <value>`.

  For example: `./devmem 0xF100000 32 0x10`

Qualcomm Confidential - May Contain Trade Secrets 2025-12-29 05:39:58 GMT hyungchul.won@lge.com

7 GPIO configuration in XBL

This section provides information about the GPIO configuration process in XBL, a software framework used for developing applications on various platforms. The code snippet provided shows how to configure and interact with a GPIO pin on SW6100.

The hardware controls the multiplexing of general-purpose input/output (GPIO) and alternate functions through the TLMM. The DT properties for TLMM are used to configure the GPIOs during boot processes.

To update the properties on the Linux host computer, edit the following file paths:

- `/boot_images/boot/Settings/Soc/Aspen/Core/SocInfra/TLMM/pinctrl.dtsi`
- `/boot_images/boot/Settings/Soc/Aspen/Core/SocInfra/TLMM/tlmm.dtsi`

The table lists the properties to configure the default value of a GPIO based on the requirements of the Qualcomm reference device.

Table 7-1 Configuration properties for GPIO default values

Property name	Property description	Data type	Possible values/value range	Device behavior
QCOM, sleep-config	GPIO configuration settings as defined in *pinctrl.dtsi*. The configurations are applied on device boot up.	UINT32	Pin direction: ▪ GPIO_INPUT–0x1 ▪ GPIO_OUTPUT–0x2 Pin pull configuration: ▪ GPIO_PULL_DOWN–0x4 ▪ GPIO_PULL_UP–0x8 ▪ GPIO_NO_PULL–0x10 ▪ GPIO_KEEPER–0x20 Pin output: ▪ GPIO_OUT_LOW–0x40 ▪ GPIO_OUT_HIGH–0x80 ▪ GPIO_PRG_YES–0x100 ▪ GPIO_PRG_NO–0x000	The default value is GPIO_INPUT ▪ GPIO_INPUT: Allows the state of an input pin to be read ▪ GPIO_OUTPUT: Controls the state of an output pin The default value is GPIO_PULL_DOWN. ▪ GPIO_PULL_DOWN: Logic 0, consider as connected to Ground ▪ GPIO_PULL_UP: Logic 1, connected to Vdd supply ▪ GPIO_NO_PULL: Floating/High Impedance state ▪ GPIO_KEEPER: Support the earlier state of the GPIO. When an SoC enters the deepest power-saving mode, this configuration is applied. The default value is GPIO_OUT_LOW. ▪ GPIO_OUT_HIGH: Logic is high, consider as connected to Vdd.

Qualcomm Confidential - May Contain Trade Secrets 2025-12-29 09:39:58 GMT hyungchulwoo@qc.com

Table 7-1 Configuration properties for GPIO default values (cont.)

Property name	Property description	Data type	Possible values/value range	Device behavior
				The default value is GPIO_PRG_NO. ▪ GPIO_PRG_YES: Ensures that any unused GPIOs remain in a low-power state after bootup Example: `(GPIO_INPUT | GPIO_PULL_DOWN | GPIO_OUT_LOW | GPIO_PRG_NO) /* PIN 10 */`
Compatible	Compatible property contains a read-only string that points to the compatible chipset.	String	–	Example: compatible = "qcom,pinctrl"
reg	Represents the read only GPIO base address and size.	UINT32	–	In a reg property tuple, the first index has the base address, and the second index has the size. Example: `reg = <0xf100000 0x100000>;`
ngpios	Number of read only GPIO pins in chipset.	UINT32	–	Example: `ngpios = <175>;`
width	Each GPIO pin has its own set of read only control registers. Width indicates pin to pin register offset.	–	–	Example: `width = <0x1000>;`
id	Hardware instance of read only GPIO pad ID.	UINT32	–	Example: `id = <0x0>;`

Qualcomm Confidential - May Contain Trade Secrets 2025-12-29 05:39:58 GMT hyungchul.won@lge.com

Table 7-1 Configuration properties for GPIO default values (cont.)

Property name	Property description	Data type	Possible values/value range	Device behavior
version	GPIO read only driver version. Driver 1.0 refers as 0x1.	UINT32	–	Example: `version = <0x1>;`
gpio-controller	Identifier to represent the read only connected device as a GPIO device.	String	–	–
phandles for pin configurations	Mux configuration that's used to configure the alternative functionality of GPIOs.	–	`sdc4_data_1: sdc4_data_1 { mux = <13 3>; };`	On bootup, the GPIO configures to alternate functionality of GPIO. Example: `sdc4_data_1: sdc4_data_1 { mux = <13 3> };` In this example, 13 refers to the GPIO number and 3 indicates the alternate function selection.

Qualcomm Confidential – May Contain Trade Secrets 2025-12-29 05:39:58 GMT hyunchul.won@lge.com

GPIO Configuration Using DAL APIs

The GPIO configuration is defined using the **DAL_GPIO_CFG** function, which takes four parameters:

- **GPIO Number**: The number of the GPIO pins to be configured (in this case, GPIO 22).
- **Pull-up/Pull-down**: The type of pull-up or pull-down resistor to be applied to the GPIO pin (in this case, no pull-up or pull-down resistor is applied).
- **Direction**: The direction of the GPIO pin (in this case, output).
- **Drive Strength**: The drive strength of the GPIO pin (in this case, 2 mA).

For example:

```
#define GPIO_22_ADDR 0xF116000 //0xF100000 + 0x1000*n where n is GPIO Number

DalDeviceHandle* hTlmm;
DALGpioSignalType board_id_gpio;
DALGpioValueType value;

/* Prepare config for GPIO_22 */
board_id_gpio = DAL_GPIO_CFG(22, 0, DAL_GPIO_OUT, DAL_GPIO_NO_PULL,
DAL_GPIO_2MA);
```

```
/* set the handle for operating GPIO */
if( hTlmm == NULL )
{
    DAL_DeviceAttach(DALDEVICEID_TLMM, &hTlmm);
    DalDevice_Open(hTlmm,DAL_OPEN_SHARED);
}

/*pull up the gpio and read the value*/
DalTlmm_ConfigGpio(hTlmm,board_id_gpio,DAL_TLMM_GPIO_ENABLE);

/*Write a value to GPIO*/
//Register Address can be found in 80-P1511-2X_E document,
out_dword(GPIO_22_ADDR, 0x2); // 0x2 for output high, if you need output low,
then change to 0x0.

/*For Reading the GPIO value*/
DalTlmm_GpioIn(hTlmm,board_id_gpio, &value); //value has GPIO value, (0th bit
is read only bit and 1st bit is RW where we should write when configured as
output.)
}
```

Qualcomm Confidential - May Contain Trade Secrets 2025-12-29 05:39:58 GMT hyungchul.won@lge.com

A References

Acronyms and terms

Acronym or term	Definition
GPIO	General purpose input output
I^2C	Inter-integrated circuit
LPI	Low-power island
OE	Output enable
SDC	Secure digital card
SDHC	Secure digital high capacity
SPI	Serial peripheral interface
SoC	System-on-Chip
TLMM	Top-level mode multiplexer
UART	Universal asynchronous receiver/transmitter

Related documents

Title	Number
Resources	
PINCTRL (Pin Control) Subsystem	

Qualcomm
Confidential - May Contain Trade Secrets
2025-12-29 05:39:58 GMT
hyung-hul.won@lge.com

LEGAL INFORMATION

Your access to and use of this material, along with any documents, software, specifications, reference board files, drawings, diagnostics and other information contained herein (collectively this "Material"), is subject to your (including the corporation or other legal entity you represent, collectively "You" or "Your") acceptance of the terms and conditions ("Terms of Use") set forth below. If You do not agree to these Terms of Use, you may not use this Material and shall immediately destroy any copy thereof.

1) Legal Notice.

This Material is being made available to You solely for Your internal use with those products and service offerings of Qualcomm Technologies, Inc. ("**Qualcomm Technologies**"), its affiliates and/or licensors described in this Material, and shall not be used for any other purposes. If this Material is marked as "**Qualcomm Internal Use Only**", no license is granted to You herein, and You must immediately (a) destroy or return this Material to Qualcomm Technologies, and (b) report Your receipt of this Material to qualcomm.support@qti.qualcomm.com. This Material may not be altered, edited, or modified in any way without Qualcomm Technologies' prior written approval, nor may it be used for any machine learning or artificial intelligence development purpose which results, whether directly or indirectly, in the creation or development of an automated device, program, tool, algorithm, process, methodology, product and/or other output. Unauthorized use or disclosure of this Material or the information contained herein is strictly prohibited, and You agree to indemnify Qualcomm Technologies, its affiliates and licensors for any damages or losses suffered by Qualcomm Technologies, its affiliates and/or licensors for any such unauthorized uses or disclosures of this Material, in whole or part.

Qualcomm Technologies, its affiliates and/or licensors retain all rights and ownership in and to this Material. No license to any trademark, patent, copyright, mask work protection right or any other intellectual property right is either granted or implied by this Material or any information disclosed herein, including, but not limited to, any license to make, use, import or sell any product, service or technology offering embodying any of the information in this Material.

THIS MATERIAL IS BEING PROVIDED "AS IS" WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESSED, IMPLIED, STATUTORY OR OTHERWISE. TO THE MAXIMUM EXTENT PERMITTED BY LAW, QUALCOMM TECHNOLOGIES, ITS AFFILIATES AND/OR LICENSORS SPECIFICALLY DISCLAIM ALL WARRANTIES OF TITLE, MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, SATISFACTORY QUALITY, COMPLETENESS OR ACCURACY, AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MOREOVER, NEITHER QUALCOMM TECHNOLOGIES, NOR ANY OF ITS AFFILIATES AND/OR LICENSORS, SHALL BE LIABLE TO YOU OR ANY THIRD PARTY FOR ANY EXPENSES, LOSSES, USE, OR ACTIONS HOWSOEVER INCURRED OR UNDERTAKEN BY YOU IN RELIANCE ON THIS MATERIAL.

Certain product kits, tools and other items referenced in this Material may require You to accept additional terms and conditions before accessing or using those items.

Technical data specified in this Material may be subject to U.S. and other applicable export control laws. Transmission contrary to U.S. and any other applicable law is strictly prohibited.

Nothing in this Material is an offer to sell any of the components or devices referenced herein.

This Material is subject to change without further notification.

In the event of a conflict between these Terms of Use and the *Website Terms of Use* on www.qualcomm.com, the *Qualcomm Privacy Policy* referenced on www.qualcomm.com, or other legal statements or notices found on prior pages of the Material, these Terms of Use will control. In the event of a conflict between these Terms of Use and any other agreement (written or click-through, including, without limitation any non-disclosure agreement) executed by You and Qualcomm Technologies or a Qualcomm Technologies affiliate and/or licensor with respect to Your access to and use of this Material, the other agreement will control.

These Terms of Use shall be governed by and construed and enforced in accordance with the laws of the State of California, excluding the U.N. Convention on International Sale of Goods, without regard to conflict of laws principles. Any dispute, claim or controversy arising out of or relating to these Terms of Use, or the breach or validity hereof, shall be adjudicated only by a court of competent jurisdiction in the county of San Diego, State of California, and You hereby consent to the personal jurisdiction of such courts for that purpose.

2) Trademark and Product Attribution Statements.

Qualcomm is a trademark or registered trademark of Qualcomm Incorporated. Arm is a registered trademark of Arm Limited (or its subsidiaries) in the U.S. and/or elsewhere. The Bluetooth® word mark is a registered trademark owned by Bluetooth SIG, Inc. Other product and brand names referenced in this Material may be trademarks or registered trademarks of their respective owners.

Snapdragon and Qualcomm branded products referenced in this Material are products of Qualcomm Technologies, Inc. and/or its subsidiaries. Qualcomm patented technologies are licensed by Qualcomm Incorporated.

Qualcomm
Confidential - May Contain Third Sacred
2025-12-29 05:39:58 GMT
hyungchul.woo@lge.com